

using the `math.floor()` function to ignore the remainder.

For the doubling column, we can do the same: start with 18, and then continue through a loop. In each iteration of the loop, we'll add double the previous entry to the doubling column, and we'll stop after this column is the same length as the halving column:

```
doubling = [n2]
while(len(doubling) < len(halving)):
    doubling.append(max(doubling) * 2)
```

Finally, let's put these two columns together in a dataframe called `half_double`:

```
import pandas as pd
half_double = pd.DataFrame(zip(halving, doubling))
```

We imported the Python module called `pandas` here. This module enables us to work with tables easily. In this case, we used the `zip` command, which, as suggested by its name, joins `halving` and `doubling` together like a zipper joins two sides of a garment together. The two sets of numbers, `halving` and `doubling`, start as independent lists, and after being zipped together and converted into a `pandas` dataframe, are stored in a table as two aligned columns, as shown in [Table 2-5](#). Since they're aligned and zipped together, we can refer to any row of [Table 2-5](#), such as the third row, and get the full row, including the elements from both `halving` and `doubling` (22 and 72). Being able to refer to and work with these rows will make it easy to remove the rows we don't want, like we did to [Table 2-5](#) to convert it to [Table 2-6](#).

Now we need to remove the rows whose entries in the `halving` column are even. We can test for evenness using the `%` (modulo) operator in Python, which returns a remainder after division. If a number `x` is odd, then `x%2` will be 1. The following line will keep only the rows of the table whose entry in the `halving` column is odd:

```
half_double = half_double.loc[half_double[0]%2 == 1,:]
```

In this case, we use the `loc` functionality in the `pandas` module to select only the rows we want. When we use `loc`, we specify which rows and columns we want to select in the square brackets (`[]`) that follow it. Inside the square brackets, we specify which rows and columns we want in order,